

Persistent Structured State: The First Minimal Requirement of Tool-Agnosticism in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 2, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to give the first of the three minimal requirements of tool-agnosticism — *persistent structured state* — a standalone operational treatment, expanding the parent foundational note (A1.05) with full architectural content for this requirement considered independently of the other two.

Abstract

The Coordination Knowledge Substrate (CKS) pattern commits to tool-agnosticism through three minimal requirements that any host environment must satisfy for a CKS substrate to be deployable on it. This note formalizes the first — that the host environment must support content that persists across reads and writes, with structure the substrate can rely on — as a standalone architectural commitment. The note specifies the requirement as a four-component conjunction (persistence across cell executions, persistence across sessions, persistence across time, and structured state with addressability), distinguishes it from the substrate-layer commitment to persistence (which depends on this requirement but is architecturally distinct), clarifies five properties the requirement does not impose, distinguishes it from four adjacent host-capability patterns commonly conflated with it, traces the downstream commitments that depend on it, names eight failure modes that violate it, and provides a four-clause operational test.

1. Why a precise standalone treatment is needed

The CKS pattern, as introduced in the source paper (§7.1, §7.4), commits to tool-agnosticism through three minimal requirements that any host environment must satisfy for a CKS substrate to be deployable on it: persistent structured state, human read/write access, and LLM access to substrate content. The parent foundational note (A1.05) establishes the three-requirement framework. This note formalizes the first — abbreviated *Requirement 1* in what follows — as a standalone architectural commitment with full operational content.

The motivation is twofold. First, host environments under modern infrastructure pressures — in-memory databases optimized for throughput, document stores optimized for schema flexibility, distributed caches optimized for scalability, ephemeral compute optimized for cost — frequently support persistence in some form without supporting it in the form Requirement 1 specifies. The resulting partial-support patterns each

produce a specific failure mode (enumerated in §6) that downstream consumers experience as a substrate-layer commitment failure but that originates in the host's failure to satisfy Requirement 1. Patentable derivations of CKS that focus on substrate hosting are therefore substantially more defensibly contested when Requirement 1 is publicly formalized as a standalone commitment.

Second, the substrate layer (per A2.08) commits to persistence as one of its four architectural commitments, alongside addressability, human-governed authority, and conflict preservation. Requirement 1 is what the *host* must provide for the *substrate-layer* commitment to be operationally realizable. The substrate's persistence commitment is the architectural property; the host's persistence capability is the operational basis. Without Requirement 1 named as standalone, the two blur, and persistence failures cannot be localized to either the substrate's design or the host's capability with the precision the architecture requires.

2. The Requirement 1 commitment, defined precisely

In the CKS pattern, a host environment satisfies Requirement 1 if and only if all of the following hold during the substrate's existence on that host.

(a) Content written to the host persists across cell executions. A cell execution that completes does not cause its substrate writes to be released. Subsequent cell executions can read content written by prior executions. This is durability across the cell-execution boundary — the most frequent atomic unit of substrate change in the CKS pattern.

(b) Content persists across sessions. Whatever notion of "session" the host uses — login session, process session, browser session, runtime session — substrate content is not tied to the lifetime of any single session and is accessible across sessions. This is durability across the session boundary.

(c) Content persists across time. Substrate content written on day T is retrievable on day $T+n$ for any reasonable n during the substrate's existence, modulo human-authored deletions or archival. This is durability across the temporal axis, and is what grants the substrate its capacity to be the source of truth (per A1.08) for coordination questions whose horizon exceeds the immediate session.

(d) Content has structure the substrate can rely on. The host supports organization of content into addressable units — entities, relationships, fields, rows, records, or equivalent — such that any particular piece of substrate content can be identified, located, and acted on as a discrete object. The structure does not need to conform to any specific schema; the substrate's schema is a property of the substrate, not of the host. The host provides the addressability and the persistence; the substrate provides the schema and the content.

The four components are jointly required. A host environment that satisfies fewer than all four fails Requirement 1, even if it provides other useful capabilities. Components (a)–(c) are the three durability axes; component (d) is the addressability axis. Failing any single component produces a failure mode (named in §6) that propagates to a specific substrate-layer commitment downstream (named in §5).

3. What Requirement 1 does NOT require

It is equally important to state what Requirement 1 *does not* require.

(a) No specific persistence technology. Relational databases, document stores, key-value stores, file systems, spreadsheets, structured-text files, version control systems — all can satisfy Requirement 1 if the four operational components hold. The architecture is technology-agnostic on persistence implementation.

(b) No specific schema. The substrate's schema is a substrate decision, not a host decision. The host must support addressable structured units; the substrate decides what units exist and how they are organized. Requirement 1 is satisfied by a wide range of schema patterns, from flat tables to complex hierarchical structures, provided the host supports the addressability the chosen schema needs.

(c) No high availability. Operational availability is a separate concern from persistence. A host that goes offline for maintenance, experiences planned downtime, or operates in batch mode can satisfy Requirement 1, provided substrate content persists across these events and is accessible when the host is operational. The temporal property of governance (per A2.07) addresses when governance is exercisable; Requirement 1 addresses what the host must support when it is operational.

(d) No specific write timing or global consistency. Asynchronous, eventually-consistent, batch-committed, or distributed write patterns can satisfy Requirement 1, provided that committed writes persist across the boundaries specified in §2 and that the substrate's source-of-truth commitment (per A1.08) is preserved through whatever reconciliation mechanism the deployment uses. Requirement 1 governs persistence of committed writes, not the timing of commit or the topology of replication.

(e) No specific access latency. Read/write performance is a deployment concern, not an architectural one. A host with multi-second access latency can satisfy Requirement 1 if the persistence guarantees hold.

4. What Requirement 1 is NOT

Requirement 1 is also distinct from four adjacent host-capability patterns commonly conflated with it. Each is a partial match for Requirement 1 that fails one or more components in a way the conflation obscures.

Not generic database persistence. Database systems provide persistence as a core feature, but database persistence does not automatically satisfy Requirement 1. A database that stores content in a form lacking substrate-addressable structure — encrypted blobs, opaque binary representations, application-level serialization to single columns — fails component (d). A database whose persistence guarantee holds only under specific operational conditions (e.g., availability of a specific replication topology) fails components (a)–(c) under conditions where the topology breaks. The architectural commitment is to the four-component conjunction, not to a category of technology.

Not document-store persistence with implicit structure. Document stores typically persist content as documents whose structure is internal to each document. Requirement 1 requires substrate-addressable structure across the substrate's content as a whole, not just within individual documents — fields within documents queryable from outside, relationships between documents traversable as substrate operations. A store that fetches whole documents quickly but cannot address internal fields without retrieving and parsing the entire document fails component (d) for substrates whose units are sub-document.

Not in-memory persistence with backup. Some systems hold content in memory for performance and persist to durable storage only periodically — write-through caches, buffered writes, snapshot-based persistence. These satisfy Requirement 1 only if the persistence guarantee holds for committed writes: once committed, a write persists across the boundaries of the four components, not just until the next snapshot. A host where committed writes can be lost between snapshots fails Requirement 1 even if it has backup mechanisms.

Not eventual-consistency models without per-write persistence. Some distributed systems use eventual consistency where writes may not be durably persisted at any specific replica until reconciliation occurs. Requirement 1 can be satisfied by such systems, but only if each committed write is persisted somewhere durably from the moment of commit. Systems where writes can be lost during the eventual-consistency window fail Requirement 1, regardless of how reconciliation is handled afterward.

5. Why Requirement 1 is load-bearing

Requirement 1 is load-bearing for several CKS commitments downstream.

The substrate-layer's persistence and addressability commitments (A2.08). The substrate commits to persistence and addressable units as two of its four architectural commitments; Requirement 1 components (a)–(c) and (d) respectively are what the host must provide for these commitments to be operationally realizable. A host failing components (a)–(c) cannot host a persistence-committed substrate; a host failing component (d) cannot host an addressability-committed substrate.

The source-of-truth commitment (A1.08). The substrate is authoritative for coordination questions across time; component (c) is what makes this temporal authority operationally realizable. Without (c), the substrate cannot be the source of truth for any question whose horizon exceeds the host's persistence window.

The path-retraceability commitment (A1.07). Retraceable paths through substrate content require that the content persist long enough for paths to be reconstructed; components (a)–(c) grant this. Without them, paths constructed at time T cannot be reliably retraced at time $T+n$.

The determinism contract (A1.10). The contract's guarantees about substrate state require that the state persist in a form readers can rely on. Substrate state that does not persist across cell executions, sessions, or time cannot satisfy guarantees about reproducible reads of that state.

The non-specialist governance commitment (A1.11). Governance through commodity tools requires that the commodity tools provide persistence; Requirement 1 specifies what kind. A "commodity tool" that does not satisfy Requirement 1 cannot serve as a substrate host even if widely available, because the resulting substrate would not support the governance commitments downstream.

6. Failure modes that violate Requirement 1

Eight host-environment patterns violate Requirement 1. Each maps to one or more of the four components in §2.

(a) Pure ephemeral memory. The host stores content in process memory only, with no durable backing; content is released when the process terminates. Components (a) and (b) fail. The source paper contrasts CKS substrates against this pattern (§6.2) when distinguishing them from ephemeral context-window memory.

(b) Session-scoped storage. The host stores content tied to specific sessions; content is released when the session ends. Component (b) fails.

(c) Cache-only storage. The host treats stored content as a cache that can be evicted under memory pressure or by policy. Content may persist temporarily but is not architecturally durable. Component (c) fails for any

content that gets evicted.

(d) Time-limited persistence. The host enforces a maximum lifetime on stored content — auto-expiration after N days, retention policies that delete content automatically. Substrate content past the lifetime threshold is lost, violating component (c). Time-limited persistence may be appropriate for some kinds of content; substrate content is not such content.

(e) Unstructured persistence. The host persists content as opaque blobs — binary files without internal structure, single-string fields holding entire substrates, compressed archives without addressable units. Content persists but lacks the structured-state component. Component (d) fails.

(f) Schema-locked persistence. The host enforces a fixed schema that the substrate cannot extend or modify. Structure exists, but it is not "the substrate can rely on" because the substrate cannot shape it to its needs. Component (d) fails for substrates whose schema needs exceed what the host's locked schema supports.

(g) Backup-only durability. The host holds content non-durably during operation and relies on periodic backups for durability. Committed writes between backups are lost on host failure. Components (a)–(c) fail intermittently — the failure manifests only on host failure between backups, which makes it difficult to diagnose.

(h) Replication without per-replica persistence. The host distributes content across replicas with eventual consistency, where replicas hold content only transiently and rely on reconciliation for durability. If all replicas lose content before reconciliation, content is lost; components (a)–(c) fail when reconciliation is incomplete.

7. Operational test

A host environment satisfies Requirement 1 if and only if all of the following are true at all times during the substrate's existence on the host:

1. Content written to the host persists across cell executions; subsequent executions can read what prior executions wrote.
2. Content persists across sessions, regardless of how the host defines session boundaries.
3. Content written on day T is retrievable on day T+n for reasonable n during the substrate's existence, modulo human-authored deletions.
4. Content is organized into addressable units that the substrate can locate, reference, and act on as discrete objects.

A host that fails any of (1)–(4) does not satisfy Requirement 1, regardless of what other persistence properties it provides. The test is at the host-capability level, not at the substrate level: Requirement 1 specifies what the host makes available; the substrate-layer commitments (per A2.08) specify what the substrate commits to making of it. The two are evaluated separately.

8. Why naming Requirement 1 as standalone matters

Implementations under pressure to use modern infrastructure patterns — in-memory databases for performance, document stores for flexibility, distributed caches for scalability, ephemeral compute for cost

— consistently drift toward host environments that partially satisfy Requirement 1. The drift is steady because each pattern has its own benefits and the substrate's needs can seem like configuration choices rather than architectural commitments. The resulting systems exhibit substrate-layer commitment failures that look like bugs in the substrate but are in fact host-capability gaps.

Naming Requirement 1 as a standalone architectural commitment — with the four components in §2, the disclaimers in §3, the four adjacent-pattern distinctions in §4, the load-bearing connections in §5, the eight failure modes in §6, and the operational test in §7 — gives downstream implementers a precise specification of what host environments must provide. Subsequent notes A2.25 and A2.26 specialize Requirements 2 and 3; A2.27 and A2.28 then formalize the joint-sufficiency and individual-necessity properties of the three-requirement set. Together the five notes give the full operational decomposition of the tool-agnosticism commitment introduced in A1.05.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Persistent Structured State: The First Minimal Requirement of Tool-Agnosticism in the Coordination Knowledge Substrate Pattern*. May 2, 2026. ORCID: 0009-0004-8065-3235.